

CAMPUSCONECTA

Diseño e implementación de una arquitectura web para una red social universitaria

Arquitectura, API REST, seguridad y persistencia

Gabriela Estefanía Cohaila Alvarado

Código 2022075746

Proyecto académico de ingeniería de software
4 de octubre de 2026

Resumen

Este artículo presenta el diseño y la implementación de CampusConecta, una plataforma web de interacción universitaria construida con ASP.NET Core, React y PostgreSQL. El trabajo aborda la separación entre interfaz, API y persistencia; el modelo relacional; la autenticación mediante JSON Web Token; la documentación OpenAPI; y las decisiones que permiten ejecutar y probar el sistema de forma reproducible. La verificación incluyó seis pruebas automáticas, compilación del frontend y un flujo integral contra PostgreSQL real. Todas las pruebas automáticas aprobaron. El flujo de aceptación creó y recuperó una publicación, un comentario y una reacción, y confirmó que la búsqueda y el panel personal reflejaban los datos persistidos. Una muestra exploratoria de veinte consultas locales al feed obtuvo una media de 5.40 ms y un percentil 95 de 9.58 ms. Estas cifras describen el entorno de desarrollo y no estiman capacidad productiva. El resultado es una base funcional que cubre las operaciones sociales requeridas y deja identificadas las mejoras necesarias para un despliegue institucional.

Palabras clave arquitectura web, ASP.NET Core, API REST, JWT, PostgreSQL, React, red social universitaria

Introducción

Las redes sociales organizan perfiles, conexiones e intercambios de contenido dentro de un sistema delimitado. Boyd y Ellison (2007) sitúan los perfiles visibles y las relaciones entre participantes entre sus elementos centrales. En un entorno universitario, esa estructura debe incorporar además comunidades académicas, roles diferenciados y controles que eviten que cualquier usuario modifique contenido ajeno. CampusConecta se diseñó con ese alcance: ofrecer un espacio común para estudiantes, docentes y personal universitario mediante una aplicación web ejecutable, no una maqueta visual.

El problema técnico consistió en integrar once capacidades funcionales sin concentrar toda la lógica en la interfaz. El sistema debía registrar usuarios, autenticar solicitudes, administrar perfiles, publicar contenido, aceptar comentarios y reacciones, gestionar comunidades, buscar información, mostrar un feed y resumir la actividad personal. También debía persistir los datos, devolver errores comprensibles y documentar la API. La solución separa estas responsabilidades para que cada capa pueda evolucionar y probarse con un impacto acotado.

Objetivo y alcance

El objetivo fue diseñar e implementar una arquitectura web completa para una red social universitaria, con una API REST segura, una interfaz moderna y una base de datos relacional. El alcance funcional comprende registro e inicio de sesión, consulta y edición de perfil, publicaciones, comentarios, tres tipos de reacción, comunidades con creación y membresía, búsqueda unificada, panel personal, feed y eliminación autorizada de contenido propio. Se añadió un endpoint de salud y una interfaz Swagger para facilitar la operación y la verificación.

El trabajo no incluye mensajería privada, notificaciones en tiempo real, recuperación de contraseña, almacenamiento de archivos ni un panel institucional avanzado. Estas exclusiones mantienen el proyecto dentro del alcance académico sin simular capacidades que no fueron implementadas.

Método de desarrollo

Se siguió un proceso incremental orientado a riesgos. Primero se definieron las entidades y relaciones que sostienen los flujos. Después se implementaron autenticación y autorización, porque el resto de las operaciones depende de una identidad verificable. A continuación se desarrollaron publicaciones, comentarios,

reacciones y comunidades. La búsqueda y el panel se añadieron sobre consultas ya validadas. Finalmente se construyó la interfaz React, se incorporaron datos iniciales y se ejecutaron pruebas automáticas y manuales contra una instancia PostgreSQL real.

1. Modelar usuarios, comunidades, membresías, publicaciones, comentarios y reacciones.
2. Implementar contratos HTTP y reglas de autorización en controladores ASP.NET Core.
3. Construir la SPA React y conectar todos los estados visibles con la API.
4. Aplicar la migración, cargar datos de demostración y verificar el flujo completo.
5. Empaquetar scripts reproducibles de instalación, inicio, detención y prueba.

Arquitectura propuesta

CampusConecta adopta una arquitectura de tres capas desplegadas. El navegador ejecuta una aplicación de página única escrita en React y TypeScript. La interfaz consume JSON mediante HTTP y conserva el token de sesión durante la demostración. La API ASP.NET Core expone los recursos, valida los contratos, aplica permisos y coordina la persistencia. Entity Framework Core traduce las operaciones del dominio a PostgreSQL mediante el proveedor Npgsql. La separación permite reemplazar la interfaz o incorporar otro cliente sin modificar el esquema público de la API.

Capa	Tecnología	Responsabilidad
Presentación	React 19, TypeScript, Vite	Navegación, formularios, estados de carga, mensajes y diseño responsivo
Servicios	ASP.NET Core 8 Web API	Rutas REST, validación, autenticación, autorización y respuestas HTTP
Persistencia	EF Core 8 y Npgsql	Modelo relacional, consultas, migraciones y carga inicial
Datos	PostgreSQL	Integridad referencial y almacenamiento persistente
Operación	Swagger y Docker Compose	Exploración de endpoints y ejecución reproducible

La API usa controladores agrupados por recurso. AuthController registra y autentica; UsersController gestiona perfiles; PostsController resuelve feed, publicaciones, comentarios y reacciones; CommunitiesController administra grupos y membresías; SearchController combina resultados; y DashboardController calcula indicadores del usuario. Un middleware global transforma excepciones conocidas en respuestas consistentes y evita exponer trazas internas al cliente.

Modelo de datos

El esquema contiene seis tablas principales. Users conserva la identidad, el rol y el hash de contraseña. Communities almacena cada grupo y su creador. CommunityMembers materializa la relación muchos a muchos entre usuarios y comunidades. Posts puede pertenecer al feed general o a una comunidad. Comments relaciona una intervención con su publicación y autor. Reactions vincula un usuario con una publicación y restringe la combinación a una reacción por usuario y publicación.

Entidad	Relaciones y restricciones
Users	Correo y nombre de usuario únicos; autor de publicaciones, comentarios y comunidades
Communities	Nombre único; creador obligatorio; colección de miembros y publicaciones
CommunityMembers	Clave compuesta lógica por usuario y comunidad; fecha de unión
Posts	Autor obligatorio; comunidad opcional; contenido y fecha de creación
Comments	Autor y publicación obligatorios; eliminación controlada por propietario
Reactions	Una reacción por usuario y publicación; tipo validado por la API

La migración InitialCreate reproduce este modelo desde el código. Las claves foráneas preservan la integridad y las restricciones únicas impiden duplicar identidades o membresías. PostgreSQL documenta las restricciones como reglas que rechazan escrituras incompatibles con el modelo; en este proyecto complementan, pero no sustituyen, las validaciones de la API.

Diseño de la API REST

Las rutas representan recursos y usan métodos HTTP según la operación. POST crea registros; GET consulta; PUT actualiza o establece una reacción; y DELETE elimina contenido o abandona una comunidad. Los códigos 201, 200 y 204 indican resultados correctos, mientras que 400 informa una solicitud inválida, 401 una sesión ausente o inválida, 403 falta de permiso y 404 un recurso inexistente. Esta distinción facilita que el frontend muestre mensajes precisos.

Área	Operaciones representativas	Protección
Autenticación	POST /api/auth/register y /login	Públicas
Perfil	GET y PUT /api/users/me	JWT
Contenido	GET/POST /api/posts; comentarios; reacciones	JWT; propiedad para eliminar
Comunidades	GET/POST; join; leave	JWT; reglas de membresía
Consulta	GET /api/search y /api/dashboard	JWT
Operación	GET /health y /swagger	Públicas en desarrollo

Autenticación y seguridad

El registro valida formato de correo, nombre, rol y fortaleza mínima de contraseña. La contraseña nunca se guarda en texto legible: PasswordHasher de ASP.NET Core produce un hash con salt y parámetros de trabajo incorporados. Esta decisión coincide con la recomendación de OWASP de utilizar funciones adaptativas con salt para el almacenamiento de contraseñas. La cuenta se autentica comparando el valor recibido con el hash almacenado.

Después del inicio de sesión, la API emite un JWT firmado que incluye el identificador, el correo, el nombre de usuario y el rol. RFC 7519 define JWT como una representación compacta de claims. El middleware valida firma, emisor, audiencia y expiración antes de construir la identidad de la solicitud. Los controladores obtienen el identificador desde los claims y comprueban propiedad o membresía antes de modificar datos. Un endpoint protegido sin token devolvió 401 en la prueba automática.

La clave incluida en la configuración es sólo para desarrollo local. Un despliegue real debe inyectar una clave aleatoria mediante un gestor de secretos, usar HTTPS, reducir la exposición de Swagger y preferir un proveedor OpenID Connect u OAuth cuando la plataforma se integre con la identidad institucional. La documentación de Microsoft advierte que los tokens creados por una aplicación son adecuados para sistemas cerrados y pruebas, pero no reemplazan por sí solos una autoridad de identidad administrada.

CampusConecta API OpenAPI 3.0

API REST de la red social universitaria CampusConecta.

Authorize

Auth

- POST /api/auth/register
- POST /api/auth/login

CampusConecta.Api

- GET /health

Communities

- GET /api/communities
- POST /api/communities
- GET /api/communities/{id}
- DELETE /api/communities/{id}
- POST /api/communities/{id}/join
- DELETE /api/communities/{id}/leave

Dashboard

- GET /api/dashboard

Posts

- GET /api/posts
- POST /api/posts
- GET /api/posts/{id}
- DELETE /api/posts/{id}
- POST /api/posts/{id}/comments
- PUT /api/posts/{id}/reaction

Search

- GET /api/search

Users

- GET /api/users/me
- PUT /api/users/me
- GET /api/users/{id}

Schemas

- AuthResponse >
- CommentRequest >
- CommentResponse >
- CommunityResponse >
- CreateCommunityRequest >
- CreatePostRequest >
- DashboardResponse >
- LoginRequest >
- PostResponse >
- ReactionRequest >
- RegisterRequest >
- SearchResponse >
- UpdateProfileRequest >
- UserProfile >
- UserSummary >

Figura 1 Contrato de la API expuesto en Swagger

Implementación del frontend

La interfaz React organiza la experiencia en vistas de inicio, feed, comunidades, búsqueda, panel y perfil. `api.ts` centraliza las solicitudes y adjunta el token Bearer. `App.tsx` administra la sesión y las acciones principales. Los formularios bloquean envíos incompletos, muestran errores devueltos por la API y actualizan el estado después de cada operación. El diseño responde a pantallas estrechas y anchas sin cambiar los contratos del backend.

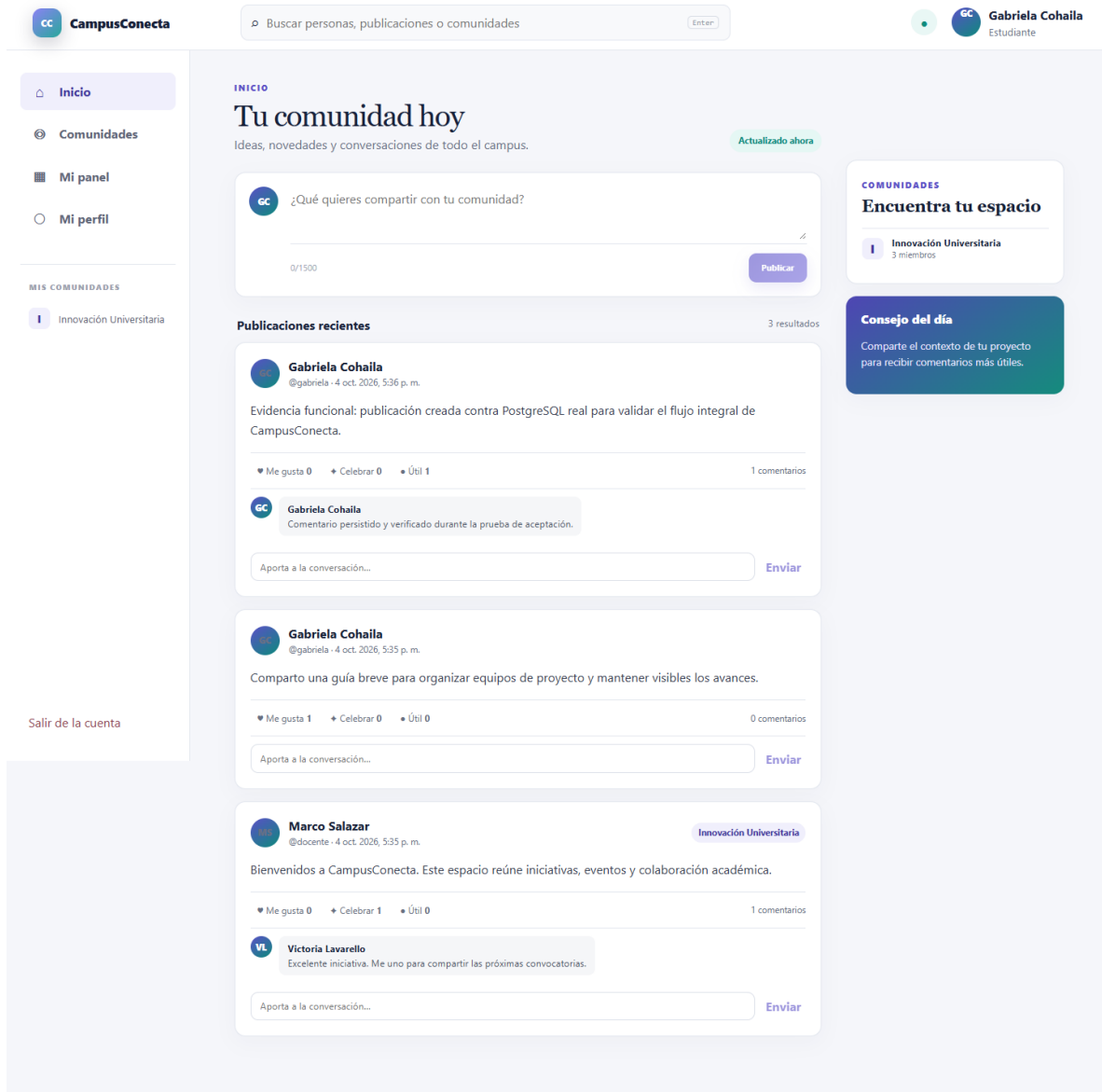


Figura 2 Feed conectado a la API con publicación, comentario y reacción persistidos

Resultados técnicos

La solución compiló en modo Release y las seis pruebas automáticas aprobaron sin casos omitidos. Las pruebas cubrieron el endpoint de salud, el hash de una cuenta inicial, la validación de contraseña, el registro con JWT, el flujo de contenido, las comunidades, la búsqueda, el perfil y la protección de rutas. El frontend superó el análisis estático y la compilación de producción. El paquete generado contenía 238.04 kB de JavaScript y 13.87 kB de CSS antes de descompresión; con gzip fueron 73.35 kB y 3.83 kB, respectivamente.

Verificación	Resultado	Evidencia
Pruebas backend	6 aprobadas, 0 fallidas, 0 omitidas	10_resultados_pruebas.txt
Análisis frontend	Aprobado sin advertencias	10_resultados_pruebas.txt
Build frontend	Producción compilada	10_resultados_pruebas.txt
Persistencia	3 usuarios, 3 posts, 2 comentarios, 3 reacciones	09_base_datos_postgresql.txt
Flujo HTTP	Login, post, comentario, reacción, búsqueda y panel correctos	Capturas 03, 05 y 07

La prueba integral se ejecutó contra PostgreSQL 18.6. Entity Framework Core aplicó la migración y el sistema conservó los datos después de las solicitudes HTTP. La entrega utiliza una imagen PostgreSQL 16 Alpine en Docker Compose para ofrecer un entorno reproducible. No se empleó una base simulada para la evidencia final de persistencia.

Se midieron veinte solicitudes consecutivas al feed en el mismo equipo. La media fue 5.40 ms, la mediana 4.73 ms y el percentil 95 9.58 ms. El mínimo fue 3.27 ms y el máximo 11.21 ms. La muestra sirve como control local de regresión; no representa usuarios concurrentes, latencia de red pública ni capacidad de producción.

Discusión

La arquitectura cumplió el objetivo porque cada requisito funcional tiene una ruta, una regla de dominio y una representación visible. La separación entre React y la API permitió probar el backend sin navegador y después confirmar la integración mediante capturas reales. El modelo relacional fue suficiente para el volumen académico y dejó las relaciones explícitas, lo que simplifica consultas y control de permisos.

Las principales limitaciones están en la operación a escala. El feed usa paginación acotada, pero todavía no incorpora cursores ni caché. Las imágenes se representan como URL y no existe un servicio de almacenamiento. La búsqueda usa coincidencias de texto sin un índice especializado. El token se gestiona de forma adecuada para la demostración, aunque una plataforma institucional debe añadir renovación segura, revocación y autenticación federada. Tampoco se realizaron pruebas de concurrencia ni auditoría externa de seguridad.

Conclusiones

CampusConecta demuestra una implementación completa y verificable de una red social universitaria. ASP.NET Core concentra contratos y permisos; PostgreSQL preserva relaciones; React presenta los flujos; JWT protege las operaciones; y Swagger permite inspeccionar la API. El resultado no depende de datos estáticos: las acciones realizadas desde HTTP modificaron la base y aparecieron en el feed, la búsqueda y el panel.

Las pruebas aportan evidencia sobre corrección funcional en el alcance desarrollado. La siguiente etapa debería priorizar identidad institucional con OpenID Connect, almacenamiento administrado de archivos, auditoría de acciones, pruebas de carga y despliegue automatizado con secretos externos. Estas mejoras extienden la base existente sin exigir una reescritura de las capas principales.

Referencias

- boyd, d. m., y Ellison, N. B. (2007). Social Network Sites: Definition, History, and Scholarship. *Journal of Computer-Mediated Communication*, 13(1), 210–230. <https://doi.org/10.1111/j.1083-6101.2007.00393.x>
- Jones, M., Bradley, J., y Sakimura, N. (2015). JSON Web Token (JWT), RFC 7519. Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc7519>
- Microsoft. (2026). Configure JWT bearer authentication in ASP.NET Core. <https://learn.microsoft.com/aspnet/core/security/authentication/configure-jwt-bearer-authentication>

OpenAPI Initiative. (2026). OpenAPI Specification. <https://spec.openapis.org/oas/>

OWASP Foundation. (2026). Password Storage Cheat Sheet.

https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

PostgreSQL Global Development Group. (2026). PostgreSQL documentation: Constraints.

<https://www.postgresql.org/docs/current/ddl-constraints.html>

React Team. (2026). React documentation. <https://react.dev/learn>